Progress Report: Implementation of GPS Graph Transformer on METR-LA Dataset

Course: MA-INF 4325 - Lab Data Science in Practice: Spatio-Temporal Data Analytics

Date: June 3, 2025

Prepared by: Arash Astanboos

University: University of Bonn (Uni-Bonn)

Overview

I have advanced the implementation of a Graph Transformer model for traffic speed prediction on the METR-LA dataset, which includes spatio-temporal data from 207 sensors in Los Angeles. This report builds on my previous work by integrating the General, Powerful, Scalable (GPS) Graph Transformer recipe, combining local message-passing (GatedGCN) with global attention (TransformerConv) and Random Walk Positional Encoding (RWPE). The objective is to enhance forecasting accuracy by leveraging the road network's graph structure and temporal dynamics.

Progress to Date

1. **Data Preparation and Loading:**
    - Loaded the METR-LA dataset, processing the adjacency matrix (shape: [207, 207]) and node values (shape: [34272, 207, 2]), extracting speed data and transposing to [207, 34272].
    - Applied MinMaxScaler for normalization, ensuring data is scaled between 0 and 1.
    - Converted the adjacency matrix to edge_index ([2, 1722]) and edge_weight tensors, optimized for GPU acceleration in PyTorch Geometric.
    - Resolved a TypeError in tensor conversion by keeping edge_index and edge_weight on CPU during NumPy operations, moving them to GPU afterward.

2. **Model Development:**
    - Implemented a GPSGraphTransformer using PyTorch Geometric, featuring a GPSLayer that combines GatedGraphConv for local message-passing and TransformerConv for global attention, with RWPE (walk_length=20) for spatial encoding.
    - Configured the model with 12 input timesteps, 64 hidden channels, 12 output timesteps, 4 layers, and 4 attention heads, followed by batch normalization and an MLP output layer.
3. **Training and Evaluation:**
    - Split the dataset into 70% training, 20% validation, and 10% test sets, with a batch size of 32.
    - Trained for 20 epochs using AdamW optimizer and MSE loss, achieving a training loss of 0.0195 and validation loss of 0.0269, indicating improved convergence over the prior model (validation loss: 0.2131).
    - Evaluated on the test set, obtaining MAE: 6.0468 mph, RMSE: 9.8066 mph, and MAPE: 8.3013%. The MAPE is significantly reduced from the previous 358045710.7407%, due to corrected metric calculations.
    - Resolved a ValueError in compute_metrics by reshaping predictions and targets to (n_samples, 207) for inverse scaling, ensuring compatibility with the MinMaxScaler.
4. **Visualization:**
    - Generated plots comparing predicted vs. actual speeds for five sensors over 12 timesteps, saved as `prediction_plot.png` in the `plots` directory.
    - Plots show improved alignment with actual values compared to the prior report, though some discrepancies persist (e.g., underprediction at peak traffic).

Challenges Encountered

- **TypeError in Data Loading**: A TypeError: can't convert cuda:0 device type tensor to numpy occurred due to premature GPU transfer of edge_index. This was fixed by performing NumPy operations on CPU tensors before moving to device.
- **ValueError in Metrics**: A ValueError: operands could not be broadcast together with shapes (12,6624) (207,) arose during inverse scaling. Resolved by

reshaping predictions to (n_samples, 207) in compute_metrics` to match the scaler's expected input.

- **High MAPE in Prior Report**: The previous MAPE of 358045710.7407% was due to numerical instability in MAPE calculation. Added a mask (target > 1.0) and epsilon (1e-2) to stabilize, reducing MAPE to 8.3013%.
- **Visualization Scaling**: Initial plots showed scaling issues; the updated `visualize_predictions` function was corrected to handle inverse transformations properly, ensuring accurate mph values.

Next Steps

- **Hyperparameter Optimization**: Tune model parameters (e.g., hidden_channels=128, num_layers=6, learning rate=0.0005) to further reduce MAE and RMSE.
- **Metric Refinement**: Investigate MAPE fluctuations by testing additional MAPE thresholds (e.g., epsilon=1e-1) and clipping outliers in target values.
- **Advanced Visualization**: Enhance plots to include confidence intervals or error bars and visualize predictions across multiple batches for comprehensive analysis.
- **Baseline Comparison**: Compare the GPS Graph Transformer with baselines (e.g., DCRNN, Graph WaveNet) to benchmark performance and include results in the final report.
- **Documentation**: Finalize a detailed report with updated metrics, visualizations, and comparisons for submission by the next milestone.

Conclusion

I have made substantial progress in implementing the GPS Graph Transformer model on the METR-LA dataset, resolving critical errors (TypeError, ValueError) and achieving promising results with a test MAE of 6.0468 mph and MAPE of 8.3013%. The integration of the GPS recipe has improved model performance compared to the prior TransformerConv-only approach. Further optimization and comparison with baselines are planned to enhance accuracy and robustness. I welcome feedback to refine my approach for the final project submission.

Attachments

`prediction_plot.png` (available in the `plots/directory` folder of the submitted notebook).